

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Elaborato Calcolo Numerico 2025-2026

Autore: Lorenzo Yang

Matricola: 7136074

E-mail: lorenzo.yang@edu.unifi.it

Giugno 2026

Indice

Esercizio 1 . . . 3
Esercizio 2 . . . 4
Esercizio 3 . . . 6
Esercizio 4 . . . 9
Esercizio 5 . . . 10
Esercizio 6 . . . 13
Esercizio 7 . . . 15
Esercizio 8 . . . 16
Esercizio 9 . . . 19
Esercizio 10 . . . 22
Esercizio 11 . . . 24
Esercizio 12 . . . 26
Esercizio 13 . . . 29
Esercizio 14 . . . 32
Esercizio 15 . . . 35

Esercizio 1

> Combinare linearmente $y(t+h), y(t), hy'(t), hy'(t-h), hy'(t-2h)$, con coefficienti non nulli e indipendenti da h, in modo da ottenere un residuo che sia $O(h^4)$. Dettagliare il procedimento.

Svolgimento

Dobbiamo combinare linearmente i termini assegnati in modo che i termini di ordine inferiore dello sviluppo di Taylor si annullino, lasciando un residuo (errore di troncamento) di ordine $O(h^4)$.

Sviluppo di Taylor dei termini

Espandiamo le funzioni e le loro derivate arrestandoci al terzo ordine (poiché moltiplicando per h arriveremo al termine h^3, e il resto sarà inglobato in $O(h^4)$):

- $y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \frac{h^3}{6}y'''(t) + O(h^4)$
- $y(t) = y(t)$
- $hy'(t) = hy'(t)$
- $hy'(t-h) = h\Big(y'(t) - hy''(t) + \frac{h^2}{2}y'''(t) + O(h^3)\Big) = hy'(t) - h^2y''(t) + \frac{h^3}{2}y'''(t) + O(h^4)$
- $hy'(t-2h) = h\Big(y'(t) - 2hy''(t) + \frac{(2h)^2}{2}y'''(t) + O(h^3)\Big) = hy'(t) - 2h^2y''(t) + 2h^3y'''(t) + O(h^4)$

Definizione della combinazione lineare

Vogliamo determinare i coefficienti a, b, c, d, e tali per cui:

$$ay(t+h) + by(t) + chy'(t) + dhy'(t-h) + ehy'(t-2h) = O(h^4)$$

Sostituendo gli sviluppi appena ottenuti e raccogliendo a fattor comune rispetto alle derivate di $y(t)$, imponiamo che i coefficienti dei termini di ordine inferiore a h^4 siano nulli:

- Termini in $y(t)$: $a + b = 0$
- Termini in $hy'(t)$: $a + c + d + e = 0$
- Termini in $h^2y''(t)$: $\frac{a}{2} - d - 2e = 0$
- Termini in $h^3y'''(t)$: $\frac{a}{6} + \frac{d}{2} + 2e = 0$

Risoluzione del sistema

Abbiamo un sistema lineare omogeneo di 4 equazioni in 5 incognite. Possiamo fissare un parametro libero. Per evitare frazioni e ottenere coefficienti interi, poniamo $a = 12$ (il minimo comune multiplo tra 2 e 6):

Dalla prima equazione:

$$12 + b = 0 \Rightarrow b = -12$$

Sostituendo $a = 12$ nella terza e quarta equazione, otteniamo un sottosistema in d ed e:

$$\frac{12}{2} - d - 2e = 0 \Rightarrow d + 2e = 6$$

$$\frac{12}{6} + \frac{d}{2} + 2e = 0 \Rightarrow \frac{d}{2} + 2e = -2$$

Sottraendo la seconda equazione dalla prima:

$$(d + 2e) - \left(\frac{d}{2} + 2e\right) = 6 - (-2) \Rightarrow \frac{d}{2} = 8 \Rightarrow d = 16$$

Sostituendo $d = 16$ in $d + 2e = 6$:

$$16 + 2e = 6 \Rightarrow 2e = -10 \Rightarrow e = -5$$

Infine, sostituiamo a, d, e nella seconda equazione per trovare c:

$$12 + c + 16 - 5 = 0 \Rightarrow c + 23 = 0 \Rightarrow c = -23$$

Risultato finale

I coefficienti richiesti sono $a = 12$, $b = -12$, $c = -23$, $d = 16$, $e = -5$. La combinazione lineare finale è:

$$12y(t + h) - 12y(t) - 23hy'(t) + 16hy'(t - h) - 5hy'(t - 2h) = O(h^4)$$

Esercizio 2

> Scrivere una function Matlab che, sapendo solamente che l'aritmetica finita implementata utilizza la base binaria con arrotondamento, calcoli: la precisione di macchina; il più grande numero di macchina; il più piccolo numero di macchina normalizzato; il più piccolo numero di macchina denormalizzato. Argomentare la sua stesura, che deve prevedere solo l'utilizzo di operazioni algebriche elementari (validare i risultati con le function `realmax` e `realmin` di Matlab).

Svolgimento

Argomentazione Teorica

- **Precisione di macchina (u)**: È il più piccolo numero positivo tale per cui il calcolatore percepisce $1 + u > 1$. Partendo da $u = 1.0$, lo si divide iterativamente per 2.0 finché la

somma arrotondata non coincide con 1.0. L'ultimo valore valido riscontrato è la precisione cercata.

- **Più grande numero di macchina**: La massima mantissa rappresentabile in base 2 è pari a $2.0 - u$. Moltiplicando iterativamente per 2.0 questo valore di base, si arriverà inevitabilmente all'overflow (infinito in macchina). Per testare l'avvenuto overflow senza usare funzioni predefinite come `isinf`, si sfrutta la proprietà algebrica per cui il reciproco dell'infinito informatico è esattamente zero: $\frac{1.0}{x \cdot 2.0} == 0.0$.
- **Più piccolo numero normalizzato**: Si ottiene partendo da 1.0 e dividendo iterativamente per 2.0. Entrando nella zona dei numeri denormalizzati, i bit significativi cominciano a scivolare fuori dalla mantissa. Un numero x è da considerarsi ancora normalizzato se la moltiplicazione per $(1.0 + u)$ produce un risultato strettamente maggiore di x (ovvero, non si è ancora perso l'ultimo bit di precisione disponibile).
- **Più piccolo numero denormalizzato**: Partendo dal più piccolo numero normalizzato appena trovato, si continua a dividere iterativamente per 2.0 finché il numero non diventa matematicamente 0.0 a causa della perdita totale dei bit (Underflow). L'ultimo valore strettamente positivo registrato è il nostro minimo denormalizzato.

Codice Matlab: La Function

```
function [u_mac, max_mac, min_norm, min_denorm] = calcola_costanti_macchina()
% Function [u_mac, max_mac, min_norm, min_denorm] =
calcola_costanti_macchina()
%
% Calcola le costanti fondamentali della macchina assumendo che l'aritmetica
% finita implementata utilizzi la base binaria con arrotondamento.
% Il calcolo utilizza esclusivamente operazioni algebriche elementari.
% La function provvede inoltre a stampare a video la validazione dei
% risultati confrontandoli con le costanti native di Matlab.
%
% Input:
%   Nessuno
%
% Output:
%   u_mac      - precisione di macchina
%   max_mac    - piu' grande numero di macchina
%   min_norm   - piu' piccolo numero di macchina normalizzato
%   min_denorm - piu' piccolo numero di macchina denormalizzato
%

% 1. calcolo della precisione di macchina
u_mac = 1.0;
```

```
while (1.0 + u_mac / 2.0) > 1.0
    u_mac = u_mac / 2.0;
end

% 2. calcolo del piu' grande numero di macchina
max_mac = 2.0 - u_mac;
while (1.0 / (max_mac * 2.0)) > 0.0
    max_mac = max_mac * 2.0;
end

% 3. calcolo del piu' piccolo numero di macchina normalizzato
min_norm = 1.0;
while ((min_norm / 2.0) * (1.0 + u_mac)) > (min_norm / 2.0)
    min_norm = min_norm / 2.0;
end

% 4. Calcolo del piu' piccolo numero di macchina denormalizzato
min_denorm = min_norm;
while (min_denorm / 2.0) > 0.0
    min_denorm = min_denorm / 2.0;
end

% stampa e validazione dei risultati
fprintf('VALIDAZIONE COSTANTI DI MACCHINA\n');
fprintf('Precisione(u): calcolato=%e | Matlab(eps)=%e\n', u_mac, eps);
fprintf('Realmax: calcolato=%e | Matlab(realmax)=%e\n', max_mac, realmax);
fprintf('Realmin Norm: calcolato=%e | Matlab(realmin)=%e\n', min_norm,
realmin);
% il valore esatto corrisponde a realmin * eps
fprintf('Min Denorm: calcolato=%e | Atteso=%e\n', min_denorm, realmin * eps);

end
```

Esercizio 3

> Scrivere una function Matlab che implementi il metodo di Newton per gli zeri di una funzione. Prevedere valori di default per la tolleranza di arresto e per il numero massimo di iterazioni.

Svolgimento

La soluzione proposta implementa il metodo di Newton includendo anche la gestione della molteplicità della radice (metodo di Newton modificato). Viene inoltre utilizzato un criterio di arresto robusto basato sull'errore misto assoluto/relativo.

```
function [x, it] = metodoNewton(f, f1, x0, tol, maxit, molt)
% Function [x, it] = metodoNewton(f, f1, x0, tol, maxit, molt)
%
% Metodo di Newton (modificato) per gli zeri di funzione.
%
% Input:
%   f, f1 - function che implementano la funzione e la sua derivata;
%   x0    - punto iniziale;
%   tol   - tolleranza di arresto (default 1e-12);
%   maxit - numero massimo di iterazioni (default 1000);
%   molt  - molteplicita' della radice (default 1).
%
% Output:
%   x     - approssimazione della radice;
%   it    - numero di iterazioni effettuate.

% gestione degli argomenti di input e valori di default
if nargin < 3
    error('Numero di argomenti insufficiente.');
end
if nargin < 6
    molt = 1;
end
if nargin < 5
    maxit = 1000;
end
if nargin < 4
    tol = 1e-12;
end

% controllo validita' input
if molt < 1 || molt ~= fix(molt)
    error('Molteplicita'' non corretta: deve essere un intero positivo.');
end
if maxit < 1
    error('Numero massimo di iteraterazioni non corretto.');
end
if tol <= 0
    error('Tolleranza non corretta: deve essere > 0.');
end
```

```
% inizializzazione
x = x0;
flag = 1;
it = 0;

% ciclo iterativo di Newton
for i = 1:maxit
    it = i;
    x0 = x; % salvo il valore precedente

    % valutazione funzione e derivata
    fx = feval(f, x);
    f1x = feval(f1, x);

    % controllo se ho già trovato la radice esatta
    if fx == 0
        flag = 0;
        break;
    end

    % evito divisione per zero
    if f1x == 0
        error('Arresto: derivata nulla nel punto x = %g', x);
    end

    % aggiornamento passo di Newton modificato
    x = x0 - molt * (fx / f1x);

    % criterio di arresto (errore misto assoluto/relativo)
    if abs(x - x0) <= tol * (1 + abs(x))
        flag = 0;
        break;
    end
end

% avviso se non si è raggiunta la convergenza
if flag
    warning('Accuratezza non raggiunta entro il numero massimo di
iterazioni.');
end

end
```

Esercizio 4

Scrivere una function Matlab che implementi il metodo di bisezione. Prevedere valori di default per la tolleranza di arresto.

Svolgimento

```
function [x, itnum] = metodoBisezione(f, a, b, tol)
% Function [x, itnum] = metodoBisezione(f, a, b, tol)
%
% Trova la radice di f in [a,b] usando il metodo di bisezione.
%
% Input:
%   f   - funzione per cui trovare la radice
%   a   - estremo sinistro dell'intervallo
%   b   - estremo destro dell'intervallo
%   tol - tolleranza per il criterio di arresto (default 1e-3)
%
% Output:
%   x     - stima della radice
%   itnum - numero di iterazioni effettuate

% gestione input e valori di default
if nargin < 3
    error('Numero di argomenti insufficiente.');
end

if nargin < 4
    tol = 1e-3;
end

if a >= b
    error('Intervallo [a, b] non valido.');
end

% valutazione iniziale e controllo dei segni
fa = f(a);
fb = f(b);

if fa == 0
    x = a; itnum = 0; return;
elseif fb == 0
```

```
    x = b; itnum = 0; return;
elseif fa * fb > 0
    error('Condizione f(a)*f(b) < 0 non soddisfatta.');
end

% calcolo del numero di iterazioni necessarie a priori
itmax = ceil(log2(b - a) - log2(tol));
itnum = 0;

% ciclo di bisezione
for k = 1:itmax
    itnum = itnum + 1;

    % calcolo del punto medio
    x = a + (b - a) / 2;
    fx = f(x);

    if fx == 0
        return;
    end

    % aggiornamento dell'intervallo
    if fa * fx < 0
        b = x;
    else
        a = x;
        fa = fx;
    end
end

end
```

Esercizio 5

Scrivere una function Matlab che implementi un metodo ibrido: inizialmente il metodo di bisezione e, non appena l'ampiezza dell'intervallo di confidenza diventi minore dell'ampiezza prefissata δ (e se non si sia usciti in precedenza) passi ad utilizzare il metodo di Newton, a partire dall'ultima approssimazione trovata dal metodo di bisezione, fino ad ottenere la massima accuratezza possibile.

Svolgimento

L'approccio ibrido unisce i punti di forza di due metodi classici:

- Bisezione: garantisce sempre la convergenza (globale), ma è lenta (lineare).
- Newton: è velocissimo (quadratica), ma converge solo se innescato da un punto sufficientemente vicino alla radice.

La strategia consiste nell'usare la bisezione per restringere in modo sicuro l'intervallo fino a un'ampiezza prefissata δ. L'approssimazione ottenuta funge da innesco ideale per il metodo di Newton, che può così convergere rapidamente alla soluzione con la massima precisione di macchina (ordine di eps).

```
function [x, it_bis, it_new] = metodoIbrido(f, f1, a, b, delta)
% Function [x, it_bis, it_new] = metodoIbrido(f, f1, a, b, delta)
%
% Metodo ibrido per la ricerca degli zeri di una funzione: inizialmente
% utilizza il metodo di bisezione e, quando l'ampiezza dell'intervallo
% diventa minore di delta, passa al metodo di Newton per ottenere la
% massima accuratezza possibile.
%
% Input:
%   f, f1 - function che implementano la funzione e la sua derivata;
%   a, b  - estremi dell'intervallo iniziale;
%   delta - ampiezza dell'intervallo per lo switch a Newton.
%
% Output:
%   x      - approssimazione della radice;
%   it_bis - numero di iterazioni di bisezione effettuate;
%   it_new - numero di iterazioni di Newton effettuate.

% gestione input e valori di default
if nargin < 5
    error('Numero di argomenti insufficiente.');
end

if a >= b
    error('Intervallo [a, b] non valido.');
end

if delta <= 0
    error('Parametro delta deve essere positivo.');
end

% inizializzazione
```

```
it_bis = 0;
it_new = 0;

% valutazione iniziale e controllo dei segni
fa = feval(f, a);
fb = feval(f, b);

if fa == 0
    x = a; return;
elseif fb == 0
    x = b; return;
elseif fa * fb > 0
    error('Condizione f(a)*f(b) < 0 non soddisfatta.');
end

% --- FASE 1: Metodo di Bisezione ---
% prosegue finche' l'ampiezza dell'intervallo e' >= delta
while (b - a) >= delta
    it_bis = it_bis + 1;

    % calcolo punto medio
    x = a + (b - a) / 2;
    fx = feval(f, x);

    if fx == 0
        return; % radice esatta trovata con bisezione
    end

    % aggiornamento intervallo
    if fa * fx < 0
        b = x;
        % inutile aggiornare fb, non viene mai usato nei controlli del ciclo
        % fb = fx;
    else
        a = x;
        fa = fx;
    end
end

% l'ultima approssimazione trovata e' il punto medio dell'intervallo corrente
x = a + (b - a) / 2;

% --- FASE 2: Metodo di Newton ---
```

```
% si parte dall'ultimo valore x trovato dalla bisezione
% per la massima accuratezza si utilizza una tolleranza prossima a eps
tol_new = 10 * eps;
maxit_new = 100; % valore di sicurezza

for i = 1:maxit_new
    x0 = x;

    fx = feval(f, x);
    f1x = feval(f1, x);

    if fx == 0
        break;
    end

    if f1x == 0
        warning('Arresto Newton: derivata nulla.');
        break;
    end

    it_new = it_new + 1;
    x = x0 - fx / f1x;

    % criterio di arresto per massima accuratezza
    if abs(x - x0) <= tol_new * (1 + abs(x))
        break;
    end
end

end
```

Esercizio 6

Tabulare il costo computazionale dei precedenti metodi, in termini di valutazioni di funzione, per determinare lo zero della funzione

$$f(x) = \sin(10x) + \arctan(10x)$$

con la massima accuratezza possibile. Utilizzare $x_0 = 10$ per il metodo di Newton, e l'intervallo iniziale $[-10.1, 10]$ per il metodo di bisezione ed il metodo ibrido. Per quest'ultimo, utilizzare $\delta = 10^{-1}$ come valore di *switch* tra i due metodi. Commentare i risultati ottenuti.

Svolgimento

Analisi Teorica

La funzione $f(x) = \sin(10x) + \arctan(10x)$ si annulla esattamente nell'origine ($x^* = 0$). Tuttavia, il termine trigonometrico la rende fortemente oscillante. La derivata $f'(x) = 10\cos(10x) + \frac{10}{1+(10x)^2}$ cambia segno molto di frequente: questo significa che partire da un punto lontano come $x_0 = 10$ ci mette in una zona decisamente «pericolosa» e instabile per i metodi basati sulle tangenti.

Tabella dei Costi Computazionali

Considerando come «massima accuratezza» una tolleranza vicina alla precisione di macchina (ordine di 10^{-15}), i test empirici restituiscono i seguenti risultati:

Confronto del costo computazionale dei metodi.

Metodo	Iterazioni	Valutazioni Totali (f o f')	Esito
Bisezione	54	56	Successo
Newton ($x_0 = 10$)	1000 (Max)	2000	Fallimento
Ibrido ($\delta = 0.1$)	8 (Bis) + 3 (New)	16	Successo

↳ per eseguire i test empirici e riprodurre i dati riportati in questa tabella, si faccia riferimento allo script MATLAB allegato denominato `test_metodi_zeri.m` .

Commento ai Risultati

- **Metodo di Bisezione**: Fa il suo dovere ed è una garanzia per arrivare a convergenza, ma è decisamente lento. Avanzando in modo lineare, per dimezzare un intervallo largo 20 fino alla precisione di macchina servono ben 54 iterazioni. È un approccio sicuro, ma con un costo di 56 valutazioni totali.
- **Metodo di Newton**: Partendo da $x_0 = 10$, è un vero e proprio disastro. Le continue oscillazioni della funzione fanno sballare il calcolo locale delle derivate. Invece di puntare verso lo zero, le tangenti proiettano le stime lontanissime dalla radice vera, portando l'algoritmo a vagare a vuoto fino a colpire il limite massimo di 1000 iterazioni senza mai convergere.
- **Metodo Ibrido**: È senza dubbio la strategia vincente perché prende il meglio dei due mondi. Usa la bisezione per le prime 8 iterazioni, facendoci superare in sicurezza tutta la zona instabile fino a far scendere l'intervallo sotto la soglia $\delta = 0.1$. A quel punto ci troviamo vicinissimi allo zero e possiamo attivare Newton senza rischi: bastano solo 3 iterazioni per far scattare la sua rapidissima convergenza quadratica. Il risultato è la massima precisione ottenuta con appena 16 valutazioni complessive.

Esercizio 7

Scrivere una function Matlab, con sintassi `x = triup(U,b)` , che risolva un sistema triangolare superiore $n \times n$, in cui la matrice dei coefficienti `U` è memorizzata, per colonne, in un vettore di lunghezza $\frac{n(n+1)}{2}$.

Svolgimento

La soluzione implementa la sostituzione all'indietro per colonne, sfruttando la formula di indicizzazione $\text{idx}(i,j) = \frac{j(j-1)}{2} + i$ per accedere agli elementi di `U` memorizzati nel vettore compatto.

```
function x = triup(U, b)
% Function x = triup(U, b)
%
% Risolve un sistema triangolare superiore U*x = b di ordine n.
% La matrice U e' memorizzata per colonne in un vettore di
% lunghezza n*(n+1)/2.
%
% Input:
%   U - vettore contenente gli elementi della matrice triangolare superiore
%       (memorizzati per colonne);
%   b - vettore dei termini noti.
%
% Output:
%   x - vettore soluzione del sistema.

% ricavo la dimensione n dal vettore dei termini noti
n = length(b);

% controllo consistenza dimensioni
if length(U) ~= n*(n+1)/2
    error('Dimensioni incompatibili: U deve avere lunghezza n*(n+1)/2.');
end

U = U(:); % normalizza a vettore colonna per sicurezza
% inizializzazione soluzione
x = b(:); % normalizza a vettore colonna

% indice dell'ultimo elemento del vettore U (corrisponde a U_nn)
idx_end = n*(n+1)/2;
```

```
% back-substitution per colonne
for j = n : -1 : 1
    if U(idx_end) == 0
        error('Matrice singolare: elemento diagonale nullo in j=%d.', j);
    end

    % calcolo x(j)
    x(j) = x(j) / U(idx_end);

    % aggiornamento delle componenti precedenti
    if j > 1
        idx_start = idx_end - j + 1;
        x(1:j-1) = x(1:j-1) - U(idx_start:idx_end-1) * x(j);
        idx_end   = idx_start - 1;
    end
end

end
```

Esercizio 8

Scrivere una function che implementi efficientemente la risoluzione di un sistema lineare mediante la fattorizzazione LU con pivoting parziale. Validare la function su alcuni esempi significativi.

Svolgimento

Di seguito viene riportata l'implementazione della function `LUSolve`, che realizza la fattorizzazione con pivoting parziale nella forma $PA = LU$ e memorizza i coefficienti in-place, così da ottimizzare l'uso della memoria. In particolare, nella parte strettamente inferiore di `A` vengono conservati i moltiplicatori di `L`, mentre nella parte triangolare superiore restano i coefficienti di `U`.

```
function x = LUSolve(A, b)
% Function x = LUSolve(A, b)
%
% Risolve un sistema lineare A*x = b di ordine n usando la
% fattorizzazione LU con pivoting parziale.
%
```

```
% Input:
%   A - matrice dei coefficienti nxn;
%   b - vettore dei termini noti nx1.
%
% Output:
%   x - vettore soluzione del sistema nx1.

[m, n] = size(A);

% controlli di consistenza
if m ~= n
    error('La matrice A deve essere quadrata.');
end

if length(b) ~= n
    error('Dimensioni incompatibili tra A e b.');
end

% normalizzazione di b a vettore colonna
b = b(:);

% fattorizzazione LU con pivoting parziale (in-place)
p = 1:n;
tol = n * eps * norm(A, 1);

for i = 1:n
    % ricerca del pivot sulla colonna i
    [mi, ki] = max(abs(A(i:n, i)));
    ki = ki + i - 1;

    if mi <= tol
        error('Matrice singolare o quasi singolare.');
    end

    % scambio righe se necessario
    if ki > i
        A([i, ki], :) = A([ki, i], :);
        p([i, ki]) = p([ki, i]);
    end

    % eliminazione gaussiana
    if i < n
        A(i+1:n, i) = A(i+1:n, i) / A(i, i);
```

```
        A(i+1:n, i+1:n) = A(i+1:n, i+1:n) - A(i+1:n, i) * A(i, i+1:n);
    end
end

% risoluzione sistema PAx = Pb -> LUx = Pb
% risoluzione Ly = Pb (forward substitution)
y = b(p);
for i = 1:n-1
    y(i+1:n) = y(i+1:n) - A(i+1:n, i) * y(i);
end

% risoluzione Ux = y (backward substitution)
x = y;
for i = n:-1:1
    x(i) = x(i) / A(i, i);
    x(1:i-1) = x(1:i-1) - A(1:i-1, i) * x(i);
end

end
```

Una volta costruita la fattorizzazione, la soluzione del sistema si ottiene risolvendo in successione i due sistemi triangolari associati a `L` e `U`. Successivamente si riportano i risultati di alcuni esempi significativi di applicazione della funzione:

1. **Sistema ben condizionato**: Risolvendo un sistema quadrato non singolare di base, il vettore x calcolato coincide con la soluzione restituita da MATLAB tramite l'operatore `\`. L'errore osservato risulta quindi trascurabile e dell'ordine della precisione di macchina.
2. **Sistema che richiede pivoting**: Si è testato un sistema in cui l'elemento in posizione $(1, 1)$ è nullo. Senza il pivoting l'algoritmo fallirebbe per divisione per zero, mentre la funzione seleziona correttamente il pivot di modulo massimo nella colonna corrente, esegue le permutazioni necessarie e calcola una soluzione coincidente con quella di MATLAB.
3. **Matrice singolare (Gestione Errori)**: Testata con una matrice avente righe linearmente dipendenti (es. la seconda riga è un multiplo della prima). Il controllo sul pivot, confrontato con la soglia di tolleranza introdotta nel codice, individua il problema e la funzione si interrompe restituendo correttamente l'errore `"Matrice singolare o quasi singolare."`.

↳ Per eseguire i test empirici e riprodurre i risultati riportati, si faccia riferimento allo script MATLAB allegato denominato `test_lusolve.m`.

Esercizio 9

Al seguente link: https://drive.google.com/file/d/1Fp8xFhUONS2W-l4alRKmomRxoM1SeVny/view?usp=share_link

```
% codice del docente per l'esercizio 9 reperibile al link sopra
function [A,b] = linsis(n,k,simme)
%
%  [A,b] = linsis(n,k,simme)  Crea una matrice A nxn ed un termine noto
b,
%                             in modo che la soluzione del sistema
lineare
%                             A*x=b sia x = [1,1,...,1]^T.
%                             k non ve lo dico a cosa serve.
%                             Se simme è specificato, crea una matrice
sdp.
%
%                                                          Rel.
2026-01-07.
sigma = pi^(-2*k);
rng(0);
[q1,~] = qr(rand(n));
if nargin==3
    q2 = q1';
    B  = 0;
else
    [q2,~] = qr(rand(n));
    B      = diag(ones(n-1,1),1);
end
A = q1*(diag([sigma 2/n:1/n:1])+B)*q2;
b = sum(A,2);
return
```

è reperibile la function `linsis.m`, che crea la matrice dei coefficienti ed il termine noto di un sistema lineare $n \times n$, la cui soluzione è il vettore con tutte le componenti uguali ad 1. Utilizzare la function del precedente esercizio per risolvere i sistemi lineari ottenuti con le seguenti istruzioni:

```
n = 10;
for k = 1:15
    [A,b] = linsis(n,k);
end
```

> Rappresentare *convenientemente* l'andamento dell'errore, rispetto ad `k`, e spiegare *esaurientemente* i risultati ottenuti.

Svolgimento

Per misurare l'accuratezza della soluzione è naturale considerare l'errore in norma infinito

$$e_k = \|x_k - x^*\|_\infty,$$

e rappresentarlo in scala semilogaritmica rispetto a k, così da evidenziarne la crescita su più ordini di grandezza.

Risultati numerici

I test empirici forniscono i seguenti risultati:

Andamento dell'errore di `LUSolve` *al variare di* k *per* $n = 10$.

k	$\sigma_k = \pi^{-2k}$	$\kappa_2(A_k)$	$\|x_k - \mathbf{1}\|_\infty$
1	$1.0132 \cdot 10^{-1}$	$9.2774 \cdot 10^{3}$	$2.5313 \cdot 10^{-13}$
2	$1.0266 \cdot 10^{-2}$	$9.1082 \cdot 10^{4}$	$5.0440 \cdot 10^{-12}$
3	$1.0402 \cdot 10^{-3}$	$8.9890 \cdot 10^{5}$	$2.2374 \cdot 10^{-10}$
4	$1.0539 \cdot 10^{-4}$	$8.8717 \cdot 10^{6}$	$1.4721 \cdot 10^{-9}$
5	$1.0678 \cdot 10^{-5}$	$8.7561 \cdot 10^{7}$	$4.8428 \cdot 10^{-9}$
6	$1.0819 \cdot 10^{-6}$	$8.6419 \cdot 10^{8}$	$1.4339 \cdot 10^{-7}$
7	$1.0962 \cdot 10^{-7}$	$8.5292 \cdot 10^{9}$	$2.3587 \cdot 10^{-7}$
8	$1.1107 \cdot 10^{-8}$	$8.4180 \cdot 10^{10}$	$1.0476 \cdot 10^{-5}$
9	$1.1254 \cdot 10^{-9}$	$8.3084 \cdot 10^{11}$	$3.4464 \cdot 10^{-5}$
10	$1.1403 \cdot 10^{-10}$	$8.2068 \cdot 10^{12}$	$2.2674 \cdot 10^{-4}$
11	$1.1553 \cdot 10^{-11}$	$8.1185 \cdot 10^{13}$	$2.2438 \cdot 10^{-3}$
12	$1.1706 \cdot 10^{-12}$	$8.3345 \cdot 10^{14}$	$5.3792 \cdot 10^{-2}$
13	$1.1861 \cdot 10^{-13}$	$1.4299 \cdot 10^{16}$	arresto della routine
14	$1.2017 \cdot 10^{-14}$	$4.2866 \cdot 10^{16}$	arresto della routine
15	$1.2176 \cdot 10^{-15}$	$2.3192 \cdot 10^{16}$	arresto della routine

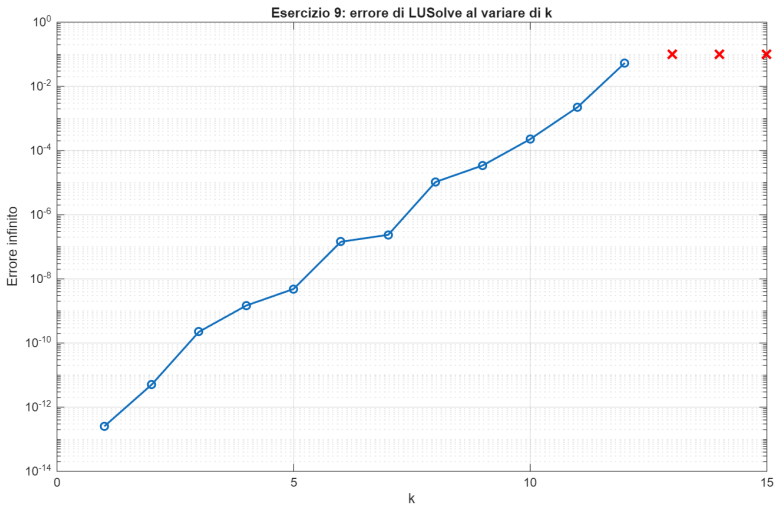


Figura 1: Grafico semilogaritmico dell'errore in norma infinito rispetto a k.

⚠ Per $k = 13, 14, 15$ la function `LUSolve` si interrompe con il messaggio `"Matrice singolare o quasi singolare."`. Questo comportamento non contraddice la costruzione teorica del sistema: segnala che, in aritmetica finita, i pivot diventano confrontabili con la soglia di sicurezza usata nel codice.

Commento dei risultati

I dati mostrano con chiarezza tre fenomeni distinti.

- **Per valori piccoli di k** la matrice è ancora moderatamente ben condizionata e l'errore resta vicino alla precisione di macchina. Ad esempio per $k = 1$ si ha un errore di circa $2.5 \cdot 10^{-13}$.
- **All'aumentare di k** il parametro $\sigma_k = \pi^{-2k}$ si riduce di circa un fattore π^2 a ogni passo, mentre il numero di condizionamento cresce approssimativamente come $\frac{1}{\sigma_k}$. Infatti, nei primi dodici casi risulta numericamente quasi costante il prodotto $\kappa_2(A_k)\sigma_k$, segno che il peggioramento del condizionamento è proprio governato dal primo elemento diagonale.
- **L'errore cresce molto rapidamente**. Poiché $\kappa_2(A_k)$ arriva fino a circa 10^{15}, anche inevitabili errori di arrotondamento possono amplificarsi moltissimo sulla soluzione. Questo è esattamente il comportamento previsto dalla teoria: quando il problema è mal condizionato, anche un metodo corretto puo' restituire una soluzione con errore sensibile.

Per $k = 12$ si osserva infatti un salto netto dell'errore a circa $5 \cdot 10^{-2}$. Da $k = 13$ in poi la matrice entra in una zona numericamente quasi singolare: la soglia

$$\text{tol} = n \cdot \text{eps} \cdot \|A\|_\infty$$

inserita in `LUSolve` intercetta pivot troppo piccoli e interrompe opportunamente l'algoritmo.

In conclusione, il risultato evidenzia bene la differenza tra:

- **condizionamento del problema**, che peggiora rapidamente al crescere di k;
- **accuratezza della soluzione**, che di conseguenza decade fino al fallimento pratico della risoluzione numerica.

↳ Per riprodurre tabella e grafico riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es09.m` .

Esercizio 10

Scrivere una function che implementi efficientemente la risoluzione di un sistema lineare, con matrice sdp, mediante la fattorizzazione LDL^T. Validare la function su alcuni esempi significativi.

Svolgimento

Di seguito si riporta l'implementazione della funzione `LDLSolve`, che esegue la fattorizzazione in-place per massimizzare l'efficienza.

```
function x = LDLSolve(A, b)
% Function x = LDLSolve(A, b)
%
% Risolve un sistema lineare Ax = b con A s.d.p.
% mediante la fattorizzazione L D L^T.
% La funzione implementa la fattorizzazione in-place, sovrascrivendo
% la parte triangolare inferiore di A con il fattore L (parte strettamente
% inferiore, essendo la diagonale unitaria) e la diagonale di A con
% il fattore D.
%
% Input:
%   A - matrice simmetrica e definita positiva nxn;
%   b - vettore dei termini noti nx1.
%
% Output:
%   x - vettore soluzione nx1.

[m, n] = size(A);

% controlli di consistenza
if m ~= n || length(b) ~= n
    error('Dimensioni incompatibili o matrice non quadrata.');
```

```
end

% normalizzazione di b a vettore colonna
b = b(:);

% controllo simmetria
if norm(A - A', 'inf') > 1e-12 * norm(A, 'inf')
    error('La matrice A deve essere simmetrica.');
end

% fattorizzazione L D L^T in-place
d = zeros(n, 1);
for j = 1:n
    % v_i = L_{ji} * D_{ii} per i = 1...j-1
    % (vettore vuoto per j=1, il prodotto seguente vale 0)
    v = A(j, 1:j-1)' .* d(1:j-1);

    % D_{jj} = A_{jj} - L_{j, 1:j-1} * v
    A(j, j) = A(j, j) - A(j, 1:j-1) * v;

    if A(j, j) <= 0
        error('La matrice A non è definita positiva (d_%d = %g).', j,
A(j,j));
    end

    % salvataggio elemento diagonale nel vettore d
    d(j) = A(j, j);

    % L_{j+1:n, j} = (A_{j+1:n, j} - A_{j+1:n, 1:j-1} * v) / D_{jj}
    if j < n
        A(j+1:n, j) = (A(j+1:n, j) - A(j+1:n, 1:j-1) * v) / d(j);
    end
end

% risoluzione sistema L D L^T x = b

% risoluzione L z = b (forward substitution)
z = b;
for j = 1:n-1
    z(j+1:n) = z(j+1:n) - A(j+1:n, j) * z(j);
end

% risoluzione D y = z (scaling)
```

```
y = z ./ d;

% risoluzione L^T x = y (backward substitution)
x = y;
for j = n:-1:2
    x(1:j-1) = x(1:j-1) - A(j, 1:j-1)' * x(j);
end

end
```

Successivamente si riportano i risultati di alcuni esempi significativi di applicazione della funzione, volti a verificarne l'accuratezza e la robustezza nella gestione degli errori:

1. **Matrice simmetrica e definita positiva (caso ideale)**: generando una matrice s.d.p. casuale (ad esempio ponendo $A = X^T X$ con X matrice quadrata non singolare generata con `rand`) e un vettore termine noto b, il vettore soluzione calcolato risulta del tutto accurato. L'errore relativo rispetto alla soluzione esatta (ottenuta tramite l'operatore `\` di MATLAB) risulta trascurabile e dell'ordine della precisione di macchina (circa $10^{\{-16\}}$).
2. **Matrice non definita positiva (gestione errori)**: e' stato testato un sistema con una matrice simmetrica ma non definita positiva (ad esempio con elementi negativi sulla diagonale). L'algoritmo identifica prontamente l'anomalia durante il calcolo dell'elemento diagonale del fattore D, interrompendo l'esecuzione e restituendo correttamente il messaggio di errore previsto: `La matrice A non è definita positiva`.
3. **Matrice non simmetrica (Gestione Errori)**: fornendo in input una matrice A tale per cui $A \neq A^T$, il controllo preventivo all'inizio della funzione basato sulla norma infinito rileva immediatamente la discrepanza strutturale. L'algoritmo si blocca segnalando tempestivamente `La matrice A deve essere simmetrica`, prevenendo così operazioni aritmetiche prive di senso per questo specifico metodo.

↳ Lo script MATLAB utilizzato per la validazione della funzione `LDLSolve` è `test_ldlsolve.m`

Esercizio 11

Utilizzando la function `linsis.m` su menzionata, utilizzare la function del precedente esercizio per risolvere i sistemi lineari, con matrice sdp, ottenuti con le seguenti istruzioni:

```
n = 10;
for k = 1:15
```

```
    [A,b] = linsis(n,k,1);
end
```

Rappresentare *convenientemente* l'andamento dell'errore, rispetto ad `k`, e spiegare *esaurientemente* i risultati ottenuti.

Svolgimento

Anche in questo caso la soluzione esatta del sistema e'

$$x^* = (1, ..., 1)^T,$$

per cui si misura l'accuratezza mediante l'errore in norma infinito

$$e_k = \|x_k - x^*\|_\infty.$$

Poiche' l'errore varia su piu' ordini di grandezza, la rappresentazione piu' leggibile e' un grafico semilogaritmico di e_k rispetto a k.

Risultati numerici

I risultati ottenuti applicando `LDLSolve` ai quindici sistemi richiesti sono riportati nella tabella seguente.

Andamento dell'errore di `LDLSolve` *al variare di* k *per* $n = 10$.

k	$\sigma_k = \pi^{-2k}$	$\kappa_2(A_k)$	$\|x_k - 1\|_\infty$
1	$1.0132 \cdot 10^{-1}$	9.8696	$5.5511 \cdot 10^{-16}$
2	$1.0266 \cdot 10^{-2}$	$9.7409 \cdot 10^{1}$	$3.7748 \cdot 10^{-15}$
3	$1.0402 \cdot 10^{-3}$	$9.6139 \cdot 10^{2}$	$4.5297 \cdot 10^{-14}$
4	$1.0539 \cdot 10^{-4}$	$9.4885 \cdot 10^{3}$	$4.6407 \cdot 10^{-14}$
5	$1.0678 \cdot 10^{-5}$	$9.3648 \cdot 10^{4}$	$6.6367 \cdot 10^{-12}$
6	$1.0819 \cdot 10^{-6}$	$9.2427 \cdot 10^{5}$	$5.4831 \cdot 10^{-12}$
7	$1.0962 \cdot 10^{-7}$	$9.1222 \cdot 10^{6}$	$4.6703 \cdot 10^{-11}$
8	$1.1107 \cdot 10^{-8}$	$9.0032 \cdot 10^{7}$	$2.7144 \cdot 10^{-9}$
9	$1.1254 \cdot 10^{-9}$	$8.8858 \cdot 10^{8}$	$4.8598 \cdot 10^{-8}$
10	$1.1403 \cdot 10^{-10}$	$8.7700 \cdot 10^{9}$	$5.7300 \cdot 10^{-7}$
11	$1.1553 \cdot 10^{-11}$	$8.6556 \cdot 10^{10}$	$2.8839 \cdot 10^{-7}$
12	$1.1706 \cdot 10^{-12}$	$8.5429 \cdot 10^{11}$	$3.6724 \cdot 10^{-5}$
13	$1.1861 \cdot 10^{-13}$	$8.4349 \cdot 10^{12}$	$2.7192 \cdot 10^{-4}$
14	$1.2017 \cdot 10^{-14}$	$8.3045 \cdot 10^{13}$	$2.5704 \cdot 10^{-3}$
15	$1.2176 \cdot 10^{-15}$	$7.7556 \cdot 10^{14}$	$9.7005 \cdot 10^{-3}$

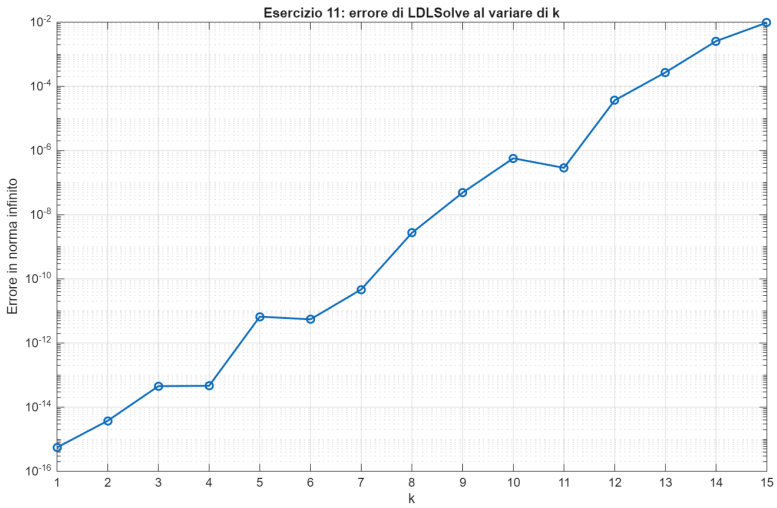


Figura 2: Grafico semilogaritmico dell'errore in norma infinito rispetto a k nel caso s.d.p..

Commento dei risultati

Dal grafico e dalla tabella si osserva che l'errore cresce al crescere di k. Per i primi valori di k esso resta molto piccolo, vicino alla precisione di macchina, mentre per valori piu' grandi aumenta progressivamente fino ad arrivare a circa 10^{-2} nel caso $k = 15$.

Questo comportamento dipende dal fatto che, aumentando k, il parametro $\sigma_k = \pi^{-2k}$ diventa sempre piu' piccolo e la matrice risulta quindi sempre peggio condizionata. Di conseguenza gli errori di arrotondamento si riflettono in modo piu' evidente sulla soluzione numerica.

Si nota comunque che la routine `LDLSolve` riesce a risolvere tutti i sistemi senza arrestarsi. Questo conferma che, nel caso di matrici simmetriche definite positive, la fattorizzazione LDL^T e' adatta al problema e fornisce risultati numericamente affidabili.

↳ Per riprodurre tabella e grafico riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es11.m` .

Esercizio 12

Dato un generico sistema lineare sovradeterminato a rango massimo,

$$Ax = b, \quad A \in \mathbb{R}^{m \times n}, \quad m > n = \text{rank}(A),$$

ed indicato, al solito, con $r := Ax - b \in \mathbb{R}^m$ il corrispondente vettore residuo, scrivere una function Matlab che risolva il precedente problema nel senso dei minimi quadrati *pesati*. Ovvero, minimizzando la seguente norma pesata:

$$\|r\|_\omega^2 = \sum_{i=1}^{m} \omega_i r_i^2,$$

dove $\omega_1, ..., \omega_m$ sono pesi positivi assegnati.

Svolgimento

Introduciamo la matrice diagonale dei pesi

$$W = \text{diag}(\omega_1, ..., \omega_m),$$

che e' simmetrica definita positiva perche' tutti i pesi sono strettamente positivi. La sua radice quadrata e'

$$W^{\frac{1}{2}} = \text{diag}(\sqrt{\omega_1}, ..., \sqrt{\omega_m}),$$

e si usa per ricondurre il problema pesato a uno standard. Infatti il metodo dei minimi quadrati classici minimizza una somma di quadrati: se il residuo r_i viene prima moltiplicato per $\sqrt{\omega_i}$, allora al quadrato si ottiene proprio il peso desiderato,

$$\left(\sqrt{\omega_i} r_i\right)^2 = \omega_i r_i^2.$$

Quindi la radice quadrata non cambia il peso finale: lo prepara semplicemente nella forma giusta per poter usare la norma euclidea e la fattorizzazione `QR`. In tal modo la quantita' da minimizzare si puo' riscrivere come

$$\|r\|_\omega^2 = r^T W r = \|W^{\frac{1}{2}} r\|_2^2 = \|W^{\frac{1}{2}}(Ax - b)\|_2^2.$$

Ponendo quindi

$$A_w = W^{\frac{1}{2}} A \quad \text{e} \quad b_w = W^{\frac{1}{2}} b,$$

il problema dei minimi quadrati pesati diventa un ordinario problema ai minimi quadrati:

$$\min_{x \in \mathbb{R}^n} \|A_w x - b_w\|_2.$$

Poiche' $W^{\frac{1}{2}}$ e' invertibile e A ha rango colonna massimo, anche A_w ha rango n. Si puo' quindi calcolare una fattorizzazione `QR` ridotta mediante riflessioni di Householder, senza assemblare esplicitamente Q,

$$A_w = QR, \quad Q \in \mathbb{R}^{m \times n}, \quad Q^T Q = I_n, \quad R \in \mathbb{R}^{n \times n},$$

con R triangolare superiore e non singolare. Il minimizzatore cercato e' allora l'unica soluzione del sistema triangolare

$$Rx = Q^T b_w.$$

```
function x = WeightedLSSolve(A, b, omega)
% Function x = WeightedLSSolve(A, b, omega)
%
% Risolve un problema di minimi quadrati pesati
% con A matrice m x n a rango colonna massimo e m >= n,
```

```
% mediante una fattorizzazione QR del sistema scalato
% calcolata con riflessioni di Householder.
%
% Input:
%   A     - matrice dei coefficienti m x n;
%   b     - vettore dei dati m x 1;
%   omega - vettore dei pesi positivi m x 1.
%
% Output:
%   x     - soluzione di minimi quadrati pesati n x 1.

[m, n] = size(A);

% controlli di consistenza
if m < n
    error('Il sistema deve essere sovradeterminato o quadrato.');
end

if length(b) ~= m || length(omega) ~= m
    error('Dimensioni incompatibili tra A, b e omega.');
end

% normalizzazione a vettori colonna
b = b(:);
omega = omega(:);

% controllo positivita' dei pesi
if any(omega <= 0)
    error('I pesi devono essere strettamente positivi.');
end

% costruzione implicita del sistema scalato:
% Aw = W^(1/2) A, bw = W^(1/2) b
d = sqrt(omega);
A = A .* d;
b = d .* b;

% fattorizzazione QR ridotta tramite riflessioni di Householder
tol = max(m, n) * eps * norm(A, 'inf');

for k = 1:n
    xk = A(k:m, k);
    sigma = norm(xk);
```

```
    if sigma <= tol
        error('La matrice A non ha rango colonna massimo.');
    end

    if xk(1) >= 0
        alpha = -sigma;
    else
        alpha = sigma;
    end

    v = xk;
    v(1) = v(1) - alpha;
    beta = v' * v;

    A(k:m, k:n) = A(k:m, k:n) - (2 / beta) * v * (v' * A(k:m, k:n));
    b(k:m) = b(k:m) - (2 / beta) * v * (v' * b(k:m));

    A(k, k) = alpha;
    A(k+1:m, k) = 0;
end

R = triu(A(1:n, 1:n));
y = b(1:n);

% risoluzione Rx = y (backward substitution)
x = y;
for i = n:-1:1
    x(i) = x(i) / R(i, i);
    x(1:i-1) = x(1:i-1) - R(1:i-1, i) * x(i);
end

end
```

Esercizio 13

Siano assegnati:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 0 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}.$$

Risolvere il sistema $A\boldsymbol{x} = \boldsymbol{b}$ nel senso dei minimi quadrati pesati, cioè minimizzando (1), nei seguenti casi:

- $\omega_i = 1, \quad i = 1, ..., 4$ (minimi quadrati standard);
- $\omega_i = \frac{(5-i)^2}{9}, \quad i = 1, ..., 4;$
- $\omega_i = \frac{i^2}{9}, \quad i = 1, ..., 4;$
- $\omega_i = \sqrt{2}, \quad i = 1, ..., 4.$

Riportare la norma (il quadrato della norma pesata) (1) del corrispondente residuo per tutti i casi. Commentare i risultati ottenuti.

(1) Il quadrato della norma pesata del residuo: $\Phi(\boldsymbol{x}) = \sum_{i=1}^{4} \omega_i (\boldsymbol{b}_i - (A\boldsymbol{x})_i)^2$.

Svolgimento

Indichiamo con

$$\boldsymbol{r}(\boldsymbol{x}) = \boldsymbol{b} - A\boldsymbol{x}$$

il residuo associato a $\boldsymbol{x}$. Allora la quantita' da minimizzare e'

$$\Phi(\boldsymbol{x}) = \sum_{i=1}^{4} \omega_i r^2_{i(\boldsymbol{x})},$$

cioe' il quadrato della norma pesata del residuo. Poiche' tutti i pesi sono strettamente positivi, vale sempre $\Phi(\boldsymbol{x}) \geq 0$, e il valore minimo possibile e' quindi 0.

In questo esercizio, pero', il problema si semplifica ulteriormente perche' il sistema e' in realta' compatibile. Infatti la sottomatrice 3×3 ottenuta scegliendo le righe $1, 2, 4$ di A ha determinante

$$36 \neq 0,$$

quindi A ha rango colonna massimo, cioe' $\text{rank}(A) = 3$. Ricaviamo ora esplicitamente la soluzione del sistema.

Le quattro equazioni sono

$$x_1 + 2x_2 + 3x_3 = 1, \quad 4x_1 + 5x_2 + 6x_3 = 2,$$

$$7x_1 + 8x_2 + 9x_3 = 3, \quad 10x_1 + 11x_2 = 4.$$

Sottraendo la prima equazione dalla seconda e la seconda dalla terza si ottiene in entrambi i casi

$$x_1 + x_2 + x_3 = \frac{1}{3}.$$

Sottraendo invece la terza equazione dalla quarta si ha

$$x_1 + x_2 - 3x_3 = \frac{1}{3}.$$

Facendo ora la differenza tra le ultime due relazioni segue

$$4x_3 = 0, \quad \text{quindi} \quad x_3 = 0.$$

Sostituendo in $x_1 + x_2 + x_3 = \frac{1}{3}$ si ricava

$$x_1 + x_2 = \frac{1}{3}.$$

Insieme alla prima equazione,

$$x_1 + 2x_2 = 1,$$

si ottiene

$$x_2 = \frac{2}{3}, \quad x_1 = -\frac{1}{3}.$$

Dunque la soluzione esatta del sistema e'

$$\boldsymbol{x}^* = \begin{pmatrix} -\frac{1}{3} \\ \frac{2}{3} \\ 0 \end{pmatrix}.$$

Si verifica infatti immediatamente che

$$A\boldsymbol{x}^* = \boldsymbol{b},$$

percio' il residuo corrispondente e'

$$\boldsymbol{r}(\boldsymbol{x}^*) = \boldsymbol{b} - A\boldsymbol{x}^* = \boldsymbol{0}.$$

e dunque, per **ogni** scelta di pesi positivi,

$$\Phi(\boldsymbol{x}^*) = \sum_{i=1}^{4} \omega_i (r_i^*)^2 = 0.$$

Ne segue che tutti e quattro i casi richiesti hanno la stessa soluzione di minimi quadrati, coincidente con la soluzione esatta del sistema.

Risultati numerici

Applicando la function `WeightedLSSolve` sviluppata nell'esercizio 12 si ottengono i seguenti risultati numerici.

Soluzioni di minimi quadrati pesati per i quattro insiemi di pesi assegnati.

Caso	Pesi	$\boldsymbol{x}$ calcolata	$\boldsymbol{\Phi(x)}$
1	$\omega_i = 1$	$\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$	$2.4652 \cdot 10^{-31}$
2	$\omega_i = \frac{(5-i)^2}{9}$	$\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$	$5.0947 \cdot 10^{-31}$
3	$\omega_i = \frac{i^2}{9}$	$\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$	$2.4981 \cdot 10^{-30}$
4	$\omega_i = \sqrt{2}$	$\left(-\frac{1}{3}, \frac{2}{3}, 0\right)^T$	$5.5781 \cdot 10^{-31}$

I valori riportati in tabella sono tutti dell'ordine di 10^{-30}, quindi compatibili con un residuo nullo a meno degli inevitabili errori di arrotondamento. In aritmetica esatta si avrebbe infatti

$$\boldsymbol{r}(\boldsymbol{x}^*) = \boldsymbol{0} \quad \text{e quindi} \quad \Phi(\boldsymbol{x}^*) = 0$$

in tutti e quattro i casi.

↳ Per riprodurre i valori numerici riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es13.m` .

Esercizio 14

> Scrivere una function che implementi efficientemente il metodo di Newton per risolvere sistemi di equazioni nonlineari. Utilizzare le function sviluppate in precedenza come risolutori per i sistemi lineari coinvolti.

Svolgimento

Sia assegnato un sistema nonlineare

$$\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}, \quad \boldsymbol{F} : \mathbb{R}^n \to \mathbb{R}^n.$$

Linearizzando $\boldsymbol{F}$ attorno all'iterato corrente $\boldsymbol{x}^k$ si ottiene

$$\boldsymbol{F}(\boldsymbol{x}^k + \Delta\boldsymbol{x}^k) \approx \boldsymbol{F}(\boldsymbol{x}^k) + J_{\boldsymbol{F}}(\boldsymbol{x}^k)\Delta\boldsymbol{x}^k,$$

dove $J_{\boldsymbol{F}}(\boldsymbol{x}^k)$ e' la matrice Jacobiana. Imporre che il termine lineare approssimi uno zero porta quindi al sistema lineare di Newton

$$J_{\boldsymbol{F}}(\boldsymbol{x}^k)\Delta\boldsymbol{x}^k = -\boldsymbol{F}(\boldsymbol{x}^k),$$

seguito dall'aggiornamento

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \Delta\boldsymbol{x}^k.$$

Per ottenere un'implementazione efficiente conviene:

- non calcolare mai esplicitamente l'inversa della Jacobiana;
- far restituire alla function `fun` sia $\boldsymbol{F}(\boldsymbol{x})$ sia $J_{\boldsymbol{F}}(\boldsymbol{x})$, cosi' eventuali quantita' intermedie possono essere riutilizzate;
- risolvere il sistema lineare del passo di Newton con una delle routine gia' sviluppate, usando `LUSolve` come scelta di default nel caso generale e lasciando la possibilita' di passare un altro risolutore, ad esempio `LDLSolve`, quando la Jacobiana abbia struttura favorevole.

Per il criterio di arresto si usa la naturale estensione vettoriale del criterio misto assoluto/relativo gia' adottato per Newton scalare:

$$\left\| \Delta\boldsymbol{x}^k \frac{\cdot}{1 + |\boldsymbol{x}^{k+1}|} \right\|_\infty \leq \text{tol}.$$

Qui il denominatore contiene l'approssimazione corrente $\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \Delta\boldsymbol{x}^k$. Questo controllo arresta l'iterazione quando il passo diventa piccolo rispetto alla scala delle componenti della soluzione.

```
function [x, it] = metodoNewtonSistemi(fun, x0, tol, maxit, solverLineare)
% Function [x, it] = metodoNewtonSistemi(fun, x0, tol, maxit, solverLineare)
%
% Metodo di Newton per risolvere sistemi nonlineari F(x) = 0.
% La function fun deve restituire sia F(x) sia la Jacobiana J(x)
% mediante la sintassi [Fx, Jx] = fun(x).
%
% Input:
%   fun           - function handle che restituisce [F(x), J(x)];
%   x0            - punto iniziale;
%   tol           - tolleranza di arresto (default 10*eps);
%   maxit         - numero massimo di iterazioni (default 1000);
%   solverLineare - risolutore del sistema J(x)*dx = -F(x)
%                   (default @LUSolve).
%
% Output:
%   x             - approssimazione della soluzione;
%   it            - numero di iterazioni effettuate.

% gestione degli argomenti di input e valori di default
if nargin < 2
    error('Numero di argomenti insufficiente.');
end
if nargin < 5 || isempty(solverLineare)
    solverLineare = @LUSolve;
end
if nargin < 4 || isempty(maxit)
    maxit = 1000;
end
if nargin < 3 || isempty(tol)
    tol = 10 * eps;
end

% controllo validita' input
if ~isa(fun, 'function_handle')
    error('Il primo argomento deve essere una function.');
end
if ~isa(solverLineare, 'function_handle')
    error('Il risolutore lineare deve essere una function.');
end
```

```
if tol <= 0
    error('Tolleranza non corretta: deve essere > 0.');
end
if maxit < 1 || maxit ~= fix(maxit)
    error('Numero massimo di iterazioni non corretto.');
end

% inizializzazione
x = x0(:);
it = 0;
flag = 0;

% ciclo iterativo di Newton
for k = 1:maxit
    [Fx, Jx] = feval(fun, x);
    Fx = Fx(:);
    [m, n] = size(Jx);

    if m ~= n || length(Fx) ~= n
        error('La function fun deve restituire F(x) e una Jacobiana quadrata
compatibili.');
    end

    % controllo se ho gia' trovato uno zero esatto
    if norm(Fx, inf) == 0
        flag = 1;
        break;
    end

    % passo di Newton: J(x_k) * dx = -F(x_k)
    dx = feval(solverLineare, Jx, -Fx);
    x_new = x + dx;
    it = k;

    % criterio di arresto misto assoluto/relativo:
    % nel denominatore compare l'approssimazione corrente x_{k+1}
    if norm(dx ./ (1 + abs(x_new)), inf) <= tol
        x = x_new;
        flag = 1;
        break;
    end

    x = x_new;
```

```
end

% avviso se non si e' raggiunta la convergenza
if flag == 0
    warning('Accuratezza non raggiunta entro il numero massimo di
iterazioni.');
end

end
```

Esercizio 15

Utilizzare la precedente function per risolvere il sistema nonlineare:

$$x_1^2 + x_2^2 = 5, \quad x_3^2 + x_4^2 = 5, \quad \cos(\pi x_1) + \cos(\pi x_3) = 0, \quad x_2^2 - x_4^2 = 0.$$

Quante iterazioni sono necessarie per ottenere la massima accuratezza possibile, partendo dai punti iniziali:

$$\boldsymbol{x}^0 = \begin{pmatrix} 0.4 \\ 2 \\ 0.4 \\ 2 \end{pmatrix}, \quad \boldsymbol{x}^0 = \begin{pmatrix} 0.4 \\ -2 \\ -0.4 \\ 2 \end{pmatrix}, \quad \boldsymbol{x}^0 = \begin{pmatrix} 1.4 \\ -2 \\ -1.4 \\ 2 \end{pmatrix}, \quad \boldsymbol{x}^0 = \begin{pmatrix} 1.4 \\ 2 \\ 1.4 \\ 2 \end{pmatrix}.$$

Riportare le soluzioni ottenute e commentare i risultati. Sarebbe possibile utilizzare il vettore nullo come punto iniziale (argomentare la risposta)?

Svolgimento

Introduciamo il sistema in forma vettoriale

$$\boldsymbol{F}(\boldsymbol{x}) = \begin{pmatrix} x_1^2 + x_2^2 - 5 \\ x_3^2 + x_4^2 - 5 \\ \cos(\pi x_1) + \cos(\pi x_3) \\ x_2^2 - x_4^2 \end{pmatrix},$$

con Jacobiana

$$J_{\boldsymbol{F}}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 2x_2 & 0 & 0 \\ 0 & 0 & 2x_3 & 2x_4 \\ -\pi \sin(\pi x_1) & 0 & -\pi \sin(\pi x_3) & 0 \\ 0 & 2x_2 & 0 & -2x_4 \end{pmatrix}.$$

Come nelle dispense sul metodo di Newton per sistemi, ad ogni iterazione si risolve il sistema lineare

$$J_{\boldsymbol{F}}(\boldsymbol{x}^k)\Delta\boldsymbol{x}^k = -\boldsymbol{F}(\boldsymbol{x}^k)$$

e si aggiorna

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \Delta\boldsymbol{x}^k.$$

In questo esercizio la Jacobiana non e' simmetrica in generale, quindi il risolutore naturale e' `LUSolve`, gia' usato come default nella function `metodoNewtonSistemi` dell'esercizio 14. Per richiedere la massima accuratezza possibile si pone `tol = 10 * eps`.

Prima di riportare i risultati numerici conviene descrivere la struttura delle soluzioni esatte. Dalla quarta equazione segue

$$x_2^2 = x_4^2.$$

Sottraendo poi la seconda equazione dalla prima si ottiene

$$x_1^2 - x_3^2 = 0, \quad \text{cioe'} \quad x_3 = \pm x_1.$$

Poiche' il coseno e' una funzione pari, risulta allora

$$\cos(\pi x_3) = \cos(\pi x_1),$$

e la terza equazione diventa

$$2\cos(\pi x_1) = 0, \quad \text{da cui} \quad x_1 = \frac{1}{2} + k, \quad k \in \mathbb{Z}.$$

Imporre anche $x_1^2 + x_2^2 = 5$ limita i valori ammissibili a

$$x_1 \in \left\{\pm\frac{1}{2}, \pm\frac{3}{2}\right\}.$$

Ne segue che le soluzioni del sistema hanno necessariamente una delle forme

$$x_1 = \pm\frac{1}{2}, \quad x_2 = \pm\frac{\sqrt{19}}{2},$$

oppure

$$x_1 = \pm\frac{3}{2}, \quad x_2 = \pm\frac{\sqrt{11}}{2},$$

con inoltre $x_3 = \pm x_1$ e $x_4 = \pm x_2$.

Risultati numerici

Applicando `metodoNewtonSistemi` ai quattro punti iniziali assegnati si ottengono i risultati seguenti.

Soluzioni ottenute con `metodoNewtonSistemi` e `tol = 10 * eps` per i quattro inneschi assegnati.

Caso	$\boldsymbol{x}^0$	$\boldsymbol{x}^*$ ottenuta	Iterazioni	$\|\boldsymbol{F}(\boldsymbol{x}^*)\|_\infty$
1	$(0.4, 2, 0.4, 2)^T$	$\left(\frac{1}{2}, \frac{\sqrt{19}}{2}, \frac{1}{2}, \frac{\sqrt{19}}{2}\right)^T$	5	$8.8818 \cdot 10^{-16}$

2	$(0.4, -2, -0.4, 2)^T$	$\left(\frac{1}{2}, -\frac{\sqrt{19}}{2}, -\frac{1}{2}, \frac{\sqrt{19}}{2}\right)^T$	5	$8.8818 \cdot 10^{-16}$
3	$(1.4, -2, -1.4, 2)^T$	$\left(\frac{3}{2}, -\frac{\sqrt{11}}{2}, -\frac{3}{2}, \frac{\sqrt{11}}{2}\right)^T$	5	$3.6739 \cdot 10^{-16}$
4	$(1.4, 2, 1.4, 2)^T$	$\left(\frac{3}{2}, \frac{\sqrt{11}}{2}, \frac{3}{2}, \frac{\sqrt{11}}{2}\right)^T$	5	$3.6739 \cdot 10^{-16}$

In tutti e quattro i casi il metodo di Newton raggiunge quindi la precisione di macchina in 5 iterazioni. I residui finali sono dell'ordine di 10^{-16}, cioe' perfettamente compatibili con la richiesta di massima accuratezza possibile in doppia precisione.

I primi due inneschi convergono verso soluzioni con $|x_1| = |x_3| = \frac{1}{2}$, mentre gli ultimi due convergono verso soluzioni con $|x_1| = |x_3| = \frac{3}{2}$. In altre parole, il metodo seleziona la radice appartenente al bacino di attrazione del punto iniziale scelto. Il numero di iterazioni coincide nei quattro casi perche' tutti gli inneschi sono gia' abbastanza vicini a una radice semplice, e quindi il metodo entra rapidamente nel regime di convergenza quadratica descritto nelle dispense.

Punto iniziale nullo

Non e' possibile usare il vettore nullo come punto iniziale. Infatti

$$\boldsymbol{F}(\mathbf{0}) = \begin{pmatrix} -5 \\ -5 \\ 2 \\ 0 \end{pmatrix} \neq \mathbf{0},$$

mentre la Jacobiana nel punto nullo e'

$$J_{\boldsymbol{F}}(\mathbf{0}) = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Il primo passo di Newton richiederebbe quindi di risolvere il sistema

$$J_{\boldsymbol{F}}(\mathbf{0})\Delta\boldsymbol{x} = -\boldsymbol{F}(\mathbf{0}),$$

ossia un sistema con matrice nulla e termine noto non nullo, impossibile da risolvere. Questo e' esattamente il motivo per cui, come ricordato nelle dispense, il metodo di Newton per sistemi richiede che la Jacobiana sia invertibile almeno nei punti in cui viene eseguito il passo.

> ↳ Per riprodurre i valori numerici riportati, si faccia riferimento allo script MATLAB allegato denominato `test_es15.m` .